Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Week 7 ()

Week 8 ()

Week 9 ()

Week 10 ()

Week 11 ()

Week 12 ()

Thank you for taking the Week 12 : Assignment 12.

Week 12 : Assignment 12

Your last recorded submission was on 2026-04-15, 20:53 Due date: 2026-04-15, 23:59 IST. IST

Understanding PageRank Through Code-Based Simulation

In the lectures, we discussed a Python program that simulates a simplified version of the PageRank algorithm using **point distribution**. The program creates a directed graph, assigns points to nodes, redistributes those points according to outgoing edges, and compares the results with NetworkX's built-in PageRank function.

1. Graph Creation
A directed graph (DiGraph) with 10 nodes (numbered 0 to 9) is created using NetworkX.

2. Random Edge Generation
The function responsible for adding edges loops over all pairs of distinct nodes (s, t) and adds a directed edge from s to t with a **50% probability**. Self-loops are avoided by checking that s != t.

3. Initial Point Assignment
Every node starts with **100 points**. These points are stored in a list where the index corresponds to the node number.

4. Point Distribution Logic
During redistribution:

- If a node has **no outgoing edges**, it keeps all its points.
- If a node has outgoing edges, its points are **split equally** among them.

5. Iterative Redistribution
The program repeatedly redistributes points over multiple rounds. After each round, the new distribution is printed. The user can stop the process by typing #.

6. Graph Visualization

Assessment submitted.
X

- Page Rank - How does Google Work ? - Part 01 (unit?unit=245&lesson=246)
- Page Rank - How does Google Work ? - Part 02 (unit?unit=245&lesson=247)
- Page Rank - How does Google Work ? - Part 03 (unit?unit=245&lesson=248)
- Page Rank - How does Google Work ? - Part 04 (unit?unit=245&lesson=249)
- Page Rank - How does Google Work ? - Part 05 (unit?unit=245&lesson=250)
- Page Rank - How does Google Work ? - Part 06 (unit?unit=245&lesson=251)
- Page Rank - How does Google Work ? - Part 07 (unit?unit=245&lesson=252)
- Page Rank - How does Google Work ? - Part 08 (unit?unit=245&lesson=253)
- Page Rank - How does Google Work ?

The graph structure, including nodes and edges, is displayed using matplotlib.

7. Ranking Nodes by Points

At the end, a ranking function stores final points in a dictionary {node: points} and sorts them in ascending order to find which nodes have the lowest and highest scores.

8. Comparison with PageRank

The program also calls NetworkX's built-in pagerank() function to compute the official PageRank values for the same graph and prints them for comparison with the manual simulation.

This simulation helps visualize how **importance flows through a network** and how PageRank emerges as a stable distribution of scores when point redistribution is repeated many times.

1) Why is a **directed graph** mandatory for implementing PageRank in the given code? ***1 point***

- (●) Because PageRank depends on the direction of influence between nodes
- ( ) Because undirected graphs cannot be read from files
- ( ) Because NetworkX only supports directed graphs
- ( ) Because visualization requires arrows

2) Why are **all nodes initially assigned equal points** before distribution begins? ***1 point***

- ( ) To speed up execution
- (●) To remove any initial bias in importance
- ( ) To match NetworkX's internal implementation
- ( ) To simplify graph drawing

3) If a node has **many outgoing edges**, how does it affect its contribution? ***1 point***

- ( ) It gains more points
- ( ) It distributes more total points
- (●) Its influence is divided among its neighbors
- ( ) It stops participating after one iteration

4) Why does the case study include a **manual Excel-style simulation**? ***1 point***

- ( ) To replace Python computation
- ( ) To compute final PageRank values
- ( ) To reduce graph complexity
- (●) To help visualize a single iteration clearly

5) Why is the PageRank process **iterative**? ***1 point***

- ( ) Because sorting requires multiple passes
- (●) Because convergence happens gradually
- ( ) Because graph size keeps changing
- ( ) Because visualization depends on iteration count

6) What is the **main purpose** of converting the points list into a dictionary? ***1 point***

Assessment submitted.
X

- Part 09 (unit?unit=245&lesson=254)
- Page Rank - How does Google Work ? - Part 10 (unit?unit=245&lesson=255)
- Page Rank - How does Google Work ? - Part 11 (unit?unit=245&lesson=256)
- Page Rank - How does Google Work ? - Part 12 (unit?unit=245&lesson=257)
- Page Rank - How does Google Work ? - Part 13 (unit?unit=245&lesson=258)
- Page Rank - How does Google Work ? - Part 14 (unit?unit=245&lesson=259)
- Page Rank - How does Google Work ? - Part 15 (unit?unit=245&lesson=260)
- Page Rank - How does Google Work ? - Part 16 (unit?unit=245&lesson=261)
- Collatz Conjecture - Part 01 (unit?unit=245&lesson=262)

- ◯ To improve numerical accuracy
- ◯ To reduce memory usage
- ◯ To speed up sorting
- ◉ To preserve node–score association

7) Which type of node can create issues in PageRank distribution? ***1 point***

- ◯ Nodes with self-loops
- ◯ Nodes with no incoming edges
- ◉ Nodes with no outgoing edges
- ◯ Nodes with high degree

8) Why is sorting performed after computing final point values? ***1 point***

- ◯ To normalize the graph
- ◉ To rank nodes by importance
- ◯ To check convergence
- ◯ To reduce floating-point error

9) What happens if the user **stops the iteration process too early**? ***1 point***

- ◯ Results remain accurate
- ◯ Values become random
- ◯ Convergence still occurs
- ◉ Rankings may be misleading

10) Why is nx.pagerank() used at the end of the case study? ***1 point***

- ◯ To replace the manual approach
- ◯ To speed up execution
- ◯ To visualize the graph
- ◉ To validate the conceptual simulation

11) What property of PageRank guarantees reliable results? ***1 point***

- ◯ Use of sorting
- ◯ Deterministic initialization
- ◉ Mathematical convergence
- ◯ Absence of cycles

12) Which factor most strongly increases a node's PageRank score? ***1 point***

- ◯ Number of outgoing edges
- ◯ Random initialization
- ◉ Links from already important nodes
- ◯ Graph visualization

13) Why is a random graph generated in the later part of the code? ***1 point***

Assessment submitted.
X

- Collatz Conjecture - Part 02 (unit?unit=245&lesson=263)
- JOC Conclusion (unit?unit=245&lesson=264)
- **Quiz: Week 12 : Assignment 12 (assessment?name=662)**
- Week 12 : Programming Assignment 1 (/noc26_cs84/progassignment?name=651)
- Week 12 : Programming Assignment 2 (/noc26_cs84/progassignment?name=652)
- Week 12 : Programming Assignment 3 (/noc26_cs84/progassignment?name=653)
- Week 12 Feedback Form: The Joy of Computing using Python (unit?unit=245&lesson=265)

Text Transcripts ()

Download Videos ()

- To replace real datasets
- To test scalability and behavior
- To guarantee convergence
- To reduce computation time

14) Why does PageRank still work even if initial points are changed? ***1 point***

- Because initial values are ignored
- Because points are normalized instantly
- Because sorting removes bias
- Because the algorithm converges to the same distribution

15) Which statement best summarizes the PageRank idea demonstrated in the case study? ***1 point***

- Popular nodes are those with many outgoing edges
- Importance is random but stabilizes
- Every node eventually gets equal rank
- A node is important if important nodes point to it

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers

Assessment submitted.
X